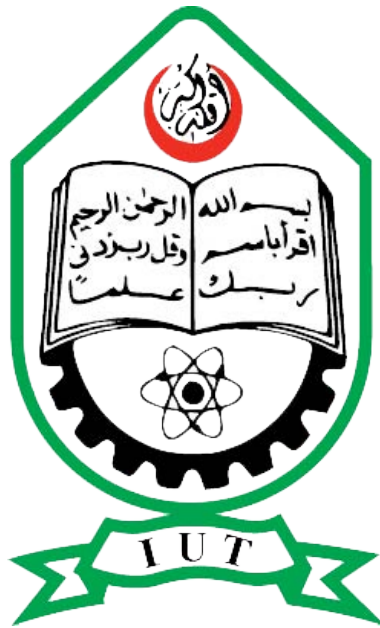




CSE 4554

Machine Learning Lab

Experiment No: 5

Name of the experiment: Understanding Convolutional Neural Networks

Dr. Hasan Mahmud

Professor, Department of CSE

Md. Tanvir Hossain Saikat

Junior Lecturer, Department of CSE

February 27, 2025

Contents

1	Objectives	4
2	Introduction to Convolutional Neural Networks (CNNs)	4
2.1	Key Components of a CNN	4
3	Architecture of a CNN	4
4	Mathematical Representation of Convolution	5
5	Pooling Operations	5
6	Activation Functions in CNNs	5
7	Training a CNN: Forward and Backward Propagation	5
8	Comparison between CNNs and Other Models	6
8.1	CNN vs Traditional Neural Networks (MLPs)	6
8.2	CNN vs Linear Models (Linear Regression, Logistic Regression)	6
8.3	CNN vs Logistic Regression	6
9	Practical Considerations	6
9.1	Overfitting in CNNs and Regularization Techniques	6
9.2	Training Time and Data Requirements	7
10	Dropout	7
10.1	How Dropout Works	7
10.2	Advantages of Dropout	7
10.3	Mathematical Representation of Dropout	7
10.4	Dropout in CNNs	7
11	Data Augmentation	8
11.1	Common Data Augmentation Techniques	8
11.2	Advantages of Data Augmentation	8
11.3	Mathematical Representation of Data Augmentation	8
11.4	Data Augmentation in CNNs	8
12	Practical Considerations for Dropout and Data Augmentation	8
12.1	Choosing Dropout Rate	8
12.2	Choosing Data Augmentation Techniques	9
13	CNN Use Cases	9
14	Traning a Convolutional Neural Network Model with Pytorch	9
15	Tasks	9
15.1	Load Dataset:	9
15.2	Downsample Dataset	9
15.3	Train a Convolutional Neural Network Model	9
15.4	Use Pooling layers	10
15.5	Use Droput layers	10
15.6	Use Augmentation Techniques	10

15.7 Use a Tensorboard to Visualize	10
---	----

1 Objectives

- To implement Convolutional Neural Networks using PyTorch
- To set up a deep learning-based image classification pipeline using CNN's
- To compare CNN-based image classification pipeline with the pipeline using only Neural Networks
- To compare CNN-based image classification accuracies after data augmentation using different strategies

2 Introduction to Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a specialized type of neural network that is primarily used for processing data that has a grid-like structure, such as images. CNNs are particularly effective in image classification, object detection, and tasks where spatial hierarchies are important.

2.1 Key Components of a CNN

CNNs are composed of various types of layers, each performing a specific function:

- **Convolutional Layer:** This is the core building block of a CNN, where a filter (or kernel) slides over the input to create feature maps by applying convolutions.
- **Pooling Layer:** This layer reduces the spatial dimensions (width and height) of the feature maps, retaining the most important information. Common pooling operations include max pooling and average pooling.
- **Fully Connected Layer:** After several convolutional and pooling layers, fully connected layers are used to make the final classification or prediction.
- **Activation Functions:** Common activation functions like ReLU, Sigmoid, and Softmax are used to introduce non-linearity into the network.

3 Architecture of a CNN

The architecture of a CNN typically consists of alternating convolutional and pooling layers followed by fully connected layers. The general structure is as follows:

- **Input Layer:** This layer takes in the raw image data, often in the form of pixel values.
- **Convolutional Layers:** These layers apply filters to the input image or the previous layer's output, extracting important features.
- **Pooling Layers:** These layers downsample the feature maps, reducing their size and improving computational efficiency.
- **Fully Connected Layers:** These are typical dense layers that make use of the features extracted by the convolutional and pooling layers to predict the output class.
- **Output Layer:** The final output is a vector of probabilities (in classification tasks) that sums to 1.

4 Mathematical Representation of Convolution

Convolution in a CNN is the mathematical operation that combines two functions (input image and filter) to produce a third function (feature map). The convolution of a filter f with an input I is given by:

$$S(i, j) = (I * f)(i, j) = \sum_m \sum_n I(m, n) \cdot f(i - m, j - n)$$

Where:

- I is the input (image or feature map),
- f is the filter (or kernel),
- S is the output feature map.

5 Pooling Operations

Pooling layers help reduce the spatial dimensions of the feature maps while retaining the most significant information. The most common types of pooling are:

- **Max Pooling:** Selects the maximum value from a patch of the feature map.
- **Average Pooling:** Computes the average value from a patch of the feature map.

Max pooling is preferred because it helps retain important features (such as edges in images) while reducing computational load.

6 Activation Functions in CNNs

As with traditional neural networks, activation functions introduce non-linearity into the network. In CNNs, common activation functions include:

- **ReLU (Rectified Linear Unit):**

$$f(z) = \max(0, z)$$

This is the most widely used activation function in CNNs due to its simplicity and efficiency.

- **Softmax:** Used in the output layer for multi-class classification tasks.

7 Training a CNN: Forward and Backward Propagation

Training a CNN follows the same principles as traditional neural networks:

- **Forward Propagation:** The input is passed through the layers of the network, and the final output is generated.
- **Loss Function:** In a classification task, the cross-entropy loss is often used to measure the difference between the predicted probabilities and the actual labels.
- **Backward Propagation (Backpropagation):** The gradients of the loss function are calculated with respect to the network's weights using the chain rule, and the weights are updated to minimize the loss.

The main difference between CNNs and traditional feedforward neural networks is how the weights (filters) in the convolutional layers are updated based on their contribution to the output error.

8 Comparison between CNNs and Other Models

8.1 CNN vs Traditional Neural Networks (MLPs)

- **Spatial Relationships:** CNNs are designed to recognize spatial hierarchies and local patterns, such as edges in an image. Traditional neural networks do not inherently recognize spatial structure.
- **Parameter Sharing:** CNNs use the same filter across the input, greatly reducing the number of parameters compared to fully connected layers, where each weight is unique.
- **Computational Efficiency:** CNNs reduce computational cost through techniques like parameter sharing and pooling, making them more efficient when dealing with high-dimensional data like images.
- **Local Receptive Fields:** CNNs focus on local patterns through small filters, while traditional networks take the entire input as a single vector.

8.2 CNN vs Linear Models (Linear Regression, Logistic Regression)

- **Non-linearity:** CNNs can model complex non-linear relationships using activation functions like ReLU, whereas linear models only capture linear relationships.
- **Feature Learning:** CNNs can automatically learn features from data (e.g., detecting edges, shapes), while linear models require manual feature engineering.
- **Hierarchical Learning:** CNNs learn hierarchical representations, starting from simple features (edges) in early layers to complex patterns (objects) in deeper layers. Linear and logistic models do not have this hierarchical structure.
- **Use Case:** CNNs are best suited for high-dimensional, spatially-structured data like images, while linear/logistic models work well for simpler, tabular data.

8.3 CNN vs Logistic Regression

Logistic regression is a linear model that is used for binary classification. The key differences between CNNs and logistic regression are:

- **Feature Representation:** Logistic regression uses raw input features, while CNNs automatically learn hierarchical features through convolution.
- **Classification Power:** CNNs can classify images with complex patterns, whereas logistic regression is limited to linearly separable data.

9 Practical Considerations

9.1 Overfitting in CNNs and Regularization Techniques

Like other models, CNNs are prone to overfitting, especially when they have many parameters. Techniques like dropout, data augmentation, and weight decay (L2 regularization) are commonly used to combat overfitting.

9.2 Training Time and Data Requirements

CNNs often require a large amount of training data to perform well, especially when the task involves complex data like images. They also require significant computational resources due to their complexity, making the use of GPUs important for efficient training.

10 Dropout

Dropout is a regularization technique that prevents overfitting by randomly "dropping out" (setting to zero) a fraction of the neurons during each training iteration. This forces the network to learn robust features that do not rely on any specific set of neurons.

10.1 How Dropout Works

In each training iteration, dropout randomly removes neurons from the network along with their connections. The probability p of retaining a neuron is typically set to 0.5 for hidden layers and closer to 1.0 for the input layer.

Mathematically, for a neuron z in the hidden layer, dropout applies the following operation during training:

$$z' = z \cdot r$$

Where r is a random variable that is 1 with probability p and 0 with probability $1 - p$. This results in:

$$z' = \begin{cases} z & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

During inference (testing), all neurons are used, but their outputs are scaled by p to account for the dropout applied during training.

10.2 Advantages of Dropout

- Reduces overfitting by ensuring that the network does not rely on any specific neurons.
- Encourages the network to learn distributed representations of data.

10.3 Mathematical Representation of Dropout

During training, the output of each hidden neuron h_i is scaled as follows:

$$h_i = \begin{cases} 0 & \text{with probability } 1 - p \\ \frac{h_i}{p} & \text{with probability } p \end{cases}$$

The scaling by $\frac{1}{p}$ ensures that the expected output remains the same, even when dropout is applied.

10.4 Dropout in CNNs

In CNNs, dropout is typically applied after the fully connected layers, although it can also be applied after convolutional layers. The value of p (retention probability) is typically chosen based on cross-validation.

11 Data Augmentation

Data Augmentation is another technique used to reduce overfitting by artificially increasing the size of the training dataset. It involves applying random transformations to the input data, such as rotations, translations, flips, and zooms, to create new training samples.

11.1 Common Data Augmentation Techniques

- **Rotation:** Rotating the image by a random angle, typically between -30 and 30 degrees.
- **Horizontal Flip:** Flipping the image horizontally.
- **Zoom:** Zooming in on the image by a random factor.
- **Translation:** Shifting the image in the horizontal or vertical direction by a random amount.
- **Shear:** Applying a shear transformation to the image.

By augmenting the data, the model sees slightly different versions of each image during training, helping it generalize better to unseen data.

11.2 Advantages of Data Augmentation

- Increases the effective size of the training dataset, which reduces overfitting.
- Helps the model become invariant to small changes in the input, such as rotations or translations.

11.3 Mathematical Representation of Data Augmentation

If X is the original image and $T(\cdot)$ is the transformation function, then the augmented image X' is given by:

$$X' = T(X)$$

Where $T(\cdot)$ can represent any transformation like rotation, zoom, or translation. For example:

- For rotation: $T(X) = \text{rotate}(X, \theta)$, where θ is a random angle.
- For zoom: $T(X) = \text{zoom}(X, z)$, where z is a random zoom factor.

11.4 Data Augmentation in CNNs

In CNNs, data augmentation is typically applied to the input images before they are passed to the network. This helps in generating new training examples from the original dataset, allowing the network to learn more robust features.

12 Practical Considerations for Dropout and Data Augmentation

12.1 Choosing Dropout Rate

The dropout rate p is a hyperparameter that must be chosen carefully. A typical value is $p = 0.5$ for the hidden layers. Too high a dropout rate can cause the network to underfit, while too low a dropout rate may not prevent overfitting effectively.

12.2 Choosing Data Augmentation Techniques

The choice of data augmentation techniques depends on the task and the dataset. For example:

- In object detection tasks, random crops and zooms are often effective.
- In medical imaging, subtle transformations like rotation and zoom are preferred to preserve the nature of the medical images.

13 CNN Use Cases

CNNs are widely used in various domains, including:

- **Image Classification:** CNNs are highly effective for image recognition tasks.
- **Object Detection:** They are used to identify objects in an image, such as in self-driving cars.
- **Medical Image Analysis:** CNNs are used to detect diseases from medical images such as MRIs or X-rays.

14 Training a Convolutional Neural Network Model with Pytorch

You can use PyTorch to build a CNN model to classify images. An example is shown in this notebook https://colab.research.google.com/github/pytorch/tutorials/blob/gh-pages/_downloads/770632dd3941d2a51b831c52ded57aa2/trainingyt.ipynb

where you train a CNN-based model on the Fashion-MNIST dataset which has the following ten classes - 'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', and 'Ankle Boot'.

For more understanding, you can refer to this video:

https://www.youtube.com/watch?v=jF43_wj_DCQ&list=PL_lsbAsL_o2CTlGHgMxNrKhZP97BaG9ZN&index=6&ab_channel=PyTorch

It is highly suggested to go through this Youtube tutorial

https://www.youtube.com/playlist?list=PL_lsbAsL_o2CTlGHgMxNrKhZP97BaG9ZN playlist if you want to better understand how to use PyTorch.

15 Tasks

15.1 Load Dataset:

Load a dataset based on your student ID (if ID%3=0 then 'CIFER10' else if ID%3=1 then 'Fashion-MNIST' else if ID%3=2 then 'MNIST')

15.2 Downsample Dataset

Sample data for only 5 classes in train, test and validation sets

15.3 Train a Convolutional Neural Network Model

Train a CNN model with 3 conv2d layers and 3 Linear layers in the end. Train for 10 epochs and a suitable learning rate. Get the accuracy on the test set.

15.4 Use Pooling layers

Use a `maxpool2d` layer after each of the `conv2d` layers. Train the model again and check the accuracy.

15.5 Use Dropout layers

This time use a dropout layer after each of the `maxpool2d` layers. Train the model again and check the accuracy.

15.6 Use Augmentation Techniques

Use data augmentation techniques to augment your training data. Train the model again and check the accuracy. You may use any two of the data augmentation strategies below:

1. `v2.AutoAugment`
2. `v2.ColorJitter`
3. `v2.functional.horizontal_flip`
4. `v2.RandomRotation`

Train for 20 epochs and a suitable learning rate. More about these data augmentation techniques can be found here (<https://pytorch.org/vision/stable/transforms.html>.)

15.7 Use a Tensorboard to Visualize

Use a tensorboard to show the loss curve of your train and validation sets in the last training task.